



Instituto Politécnico Nacional
Escuela Superior de Cómputo
Arquitectura de computadoras

Practica 4
Memoria de Datos

Profesor:

María Elena Aguilar Jauregui

Alumnos:

Lechuga Canales Héctor Jair

García Quiroz Gustavo Ivan

Eduardo Alfonso Rivera Morelos

Quintero Maldonado Fabián

Grupo:

5CV1

Fecha de entrega: 23/05/2025

Índice

1	Introducción.....	3
2	Objetivos	5
2.1	Objetivos Particulares	5
3	Metodología	6
3.1	Explicación del Proceso	6
4	Herramientas y Recursos Utilizados	8
5	Resultados	9
5.1	Resultados de Simulación.....	10
6	Conclusiones.....	12
7	Referencias	14
8	Anexo	15

1 Introducción

En el ámbito de los sistemas digitales y la arquitectura de computadoras, las memorias constituyen uno de los componentes fundamentales para el almacenamiento y procesamiento de información. Entre los diferentes tipos de memoria existentes, la memoria ROM (Read-Only Memory) desempeña un papel crucial al proporcionar almacenamiento permanente de datos que pueden ser accedidos de manera rápida y eficiente por los sistemas digitales.

Esta práctica se centra en el diseño e implementación de una memoria ROM de 16 posiciones por 8 bits (16x8) utilizando el lenguaje de descripción de hardware VHDL (VHSIC Hardware Description Language). Esta implementación permite comprender los principios fundamentales del funcionamiento de las memorias de solo lectura, incluyendo conceptos como direccionamiento, almacenamiento de datos predefinidos y acceso sincronizado mediante señales de reloj.

El desarrollo de esta práctica involucra el uso de herramientas de diseño digital profesionales, específicamente Quartus II de Intel (anteriormente Altera), para la síntesis y programación del diseño en un dispositivo FPGA (Field-Programmable Gate Array). La implementación física se realizó en la tarjeta de desarrollo Terasic DE2-115, que incorpora un FPGA Cyclone IV EP4CE115F29C7, proporcionando una plataforma robusta para la validación del diseño.

La memoria ROM implementada cuenta con una arquitectura que incluye un bus de direcciones de 4 bits, capaz de seleccionar entre 16 posiciones diferentes (direcciones 0 a 15), y un bus de datos de salida de 8 bits que proporciona el contenido almacenado en cada posición. El diseño incorpora sincronización por flanco de reloj, asegurando un funcionamiento estable y predecible del sistema.

Esta práctica no solo permite consolidar los conocimientos teóricos sobre memorias digitales, sino que también proporciona experiencia práctica en el uso de

herramientas de diseño digital, programación en VHDL, y validación de sistemas mediante testbenches y implementación en hardware real.

2 Objetivos

Diseñar e implementar una memoria ROM (Read-Only Memory) de 16 posiciones por 8 bits utilizando VHDL, con el propósito de comprender los principios fundamentales del funcionamiento de las memorias de solo lectura en sistemas digitales, mediante su síntesis, simulación y programación en un dispositivo FPGA Cyclone IV de la tarjeta DE2-115.

2.1 Objetivos Particulares

- Implementar una memoria ROM funcional que demuestre el correcto manejo de arrays de datos, direccionamiento y acceso sincronizado mediante VHDL.
- Crear y ejecutar un testbench completo que verifique el correcto funcionamiento de la ROM para todas las direcciones posibles, asegurando la integridad de los datos almacenados.
- Configurar correctamente las asignaciones de pines, estándares de E/S y restricciones de diseño para programar exitosamente el FPGA y validar el funcionamiento en la tarjeta DE2-115.

3 Metodología

La implementación de la memoria ROM 16x8 se desarrolló siguiendo una metodología estructurada que abarca desde el diseño conceptual hasta la validación en hardware real. El proceso se organizó en etapas secuenciales que garantizan un desarrollo sistemático y una verificación completa del sistema.

3.1 Explicación del Proceso

El desarrollo de la práctica se ejecutó mediante las siguientes fases:

Fase 1: Análisis y Diseño Conceptual

Se establecieron los requerimientos del sistema, definiendo una memoria ROM con capacidad de 16 posiciones de 8 bits cada una. Se determinó la arquitectura del sistema, incluyendo el bus de direcciones de 4 bits (para direccionar $2^4 = 16$ posiciones) y el bus de datos de salida de 8 bits. Se definieron los datos a almacenar en cada posición, estableciendo un patrón incremental desde 0x01 hasta 0x10 para facilitar la verificación.

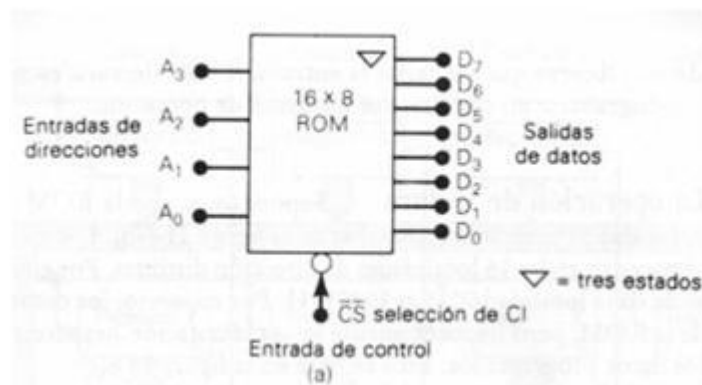


Figura 1 Diagrama de una memoria ROM

Fase 2: Implementación en VHDL

Se desarrolló el código VHDL para la entidad ROM_16x8, definiendo los puertos de entrada (clk, addr) y salida (data_out). Se implementó la arquitectura utilizando un tipo de array personalizado (rom_array) para representar la memoria. Se configuraron atributos de síntesis específicos (rom_style = "distributed") para optimizar la implementación en LUTs distribuidas del FPGA. Se implementó un

proceso síncrono con el flanco de subida del reloj para garantizar estabilidad en la lectura de datos.

Fase 3: Desarrollo del Testbench

Se creó un testbench completo (TB_ROM_16x8) para validar el funcionamiento de la ROM. El testbench incluye generación de señal de reloj con período de 10 ns, pruebas sistemáticas de todas las 16 direcciones posibles mediante un bucle for, y pruebas adicionales con direcciones específicas para verificar casos particulares. Se configuró una duración total de simulación de 200 ns para permitir múltiples ciclos de prueba.

Fase 4: Simulación y Verificación

Se ejecutó la simulación en ModelSim-Altera para verificar el comportamiento temporal del diseño. Se analizaron las formas de onda resultantes para confirmar que cada dirección produce la salida de datos correcta. Se verificó la sincronización correcta con la señal de reloj y se validó que no existen condiciones de carrera o estados indefinidos.

Fase 5: Configuración del Proyecto Quartus

Se configuró el proyecto en Quartus II especificando el dispositivo objetivo (EP4CE115F29C7) y la familia (Cyclone IV E). Se establecieron las asignaciones de pines físicos: interruptores SW0-SW3 para el bus de direcciones, LEDs verdes LEDG0-LEDG7 para el bus de datos de salida, y el reloj de 50MHz de la tarjeta (PIN_Y2). Se configuraron los estándares de E/O apropiados (3.3-V LVTTTL) para garantizar compatibilidad con la tarjeta DE2-115.

Fase 6: Síntesis e Implementación

Se ejecutó el proceso de síntesis para convertir el código VHDL en una netlist optimizada. Se realizó el proceso de place and route para mapear el diseño en los recursos físicos del FPGA. Se generó el archivo de programación (.sof) necesario para configurar el dispositivo.

Fase 7: Programación y Validación en Hardware

Se programó el FPGA utilizando el Quartus Programmer y se validó el funcionamiento físico manipulando los interruptores para cambiar las direcciones y observando las respuestas correspondientes en los LEDs. Se verificó que cada

combinación de los 4 interruptores produce el patrón de LEDs esperado según los datos almacenados en la ROM.

4 Herramientas y Recursos Utilizados

Software:

- Quartus II 13.1.0 Build 162 Web Edition
- ModelSim-Altera

Hardware:

- Tarjeta de desarrollo Terasic DE2-115: Plataforma de prototipado con FPGA Cyclone IV

5 Resultados

La implementación de la memoria ROM 16x8 se realizó exitosamente utilizando VHDL con las siguientes características técnicas:

Arquitectura del Sistema:

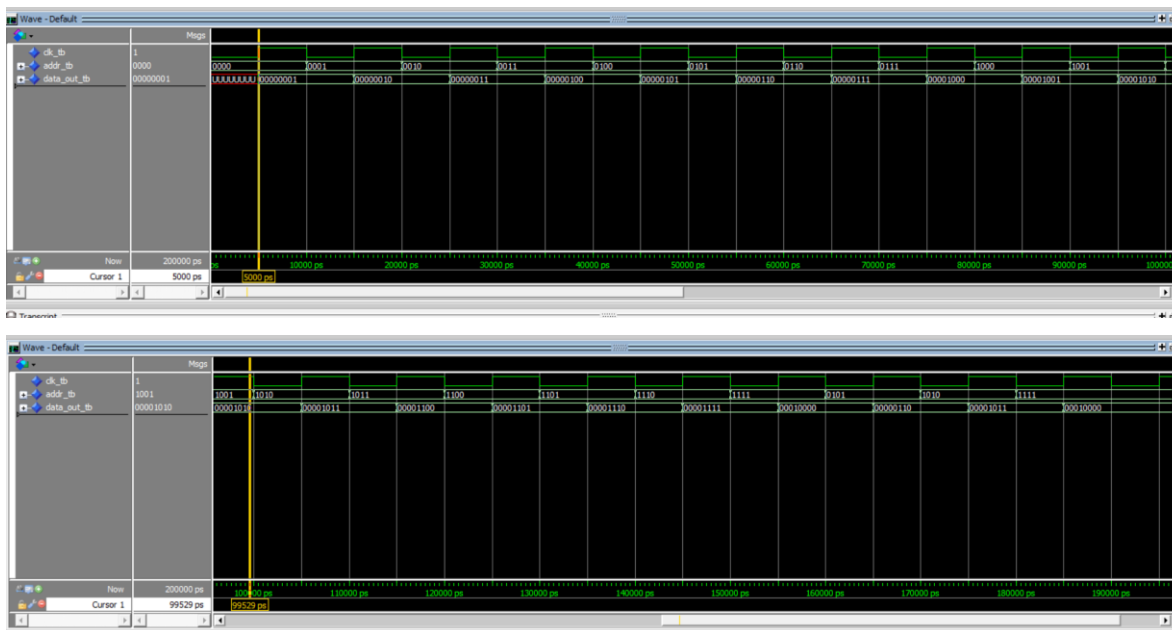
- Capacidad de almacenamiento: 16 posiciones $\times$ 8 bits = 128 bits totales
- Bus de direcciones: 4 bits (permite direccionar $2^4 = 16$ posiciones)
- Bus de datos de salida: 8 bits
- Funcionamiento: Síncrono con flanco de subida del reloj

La ROM fue inicializada con valores hexadecimales secuenciales para facilitar la verificación:

Dirección (Binario)	Dirección (Decimal)	Dato Almacenado (Hex)	Dato Almacenado (Decimal)
0000	0	0x01	1
0001	1	0x02	2
0010	2	0x03	3
0011	3	0x04	4
0100	4	0x05	5
0101	5	0x06	6
0110	6	0x07	7
0111	7	0x08	8
1000	8	0x09	9
1001	9	0x0A	10
1010	10	0x0B	11
1011	11	0x0C	12
1100	12	0x0D	13
1101	13	0x0E	14
1110	14	0x0F	15
1111	15	0x10	16

5.1 Resultados de Simulación

La validación del diseño mediante el testbench TB_ROM_16x8.vhd demostró el correcto funcionamiento de la memoria ROM:

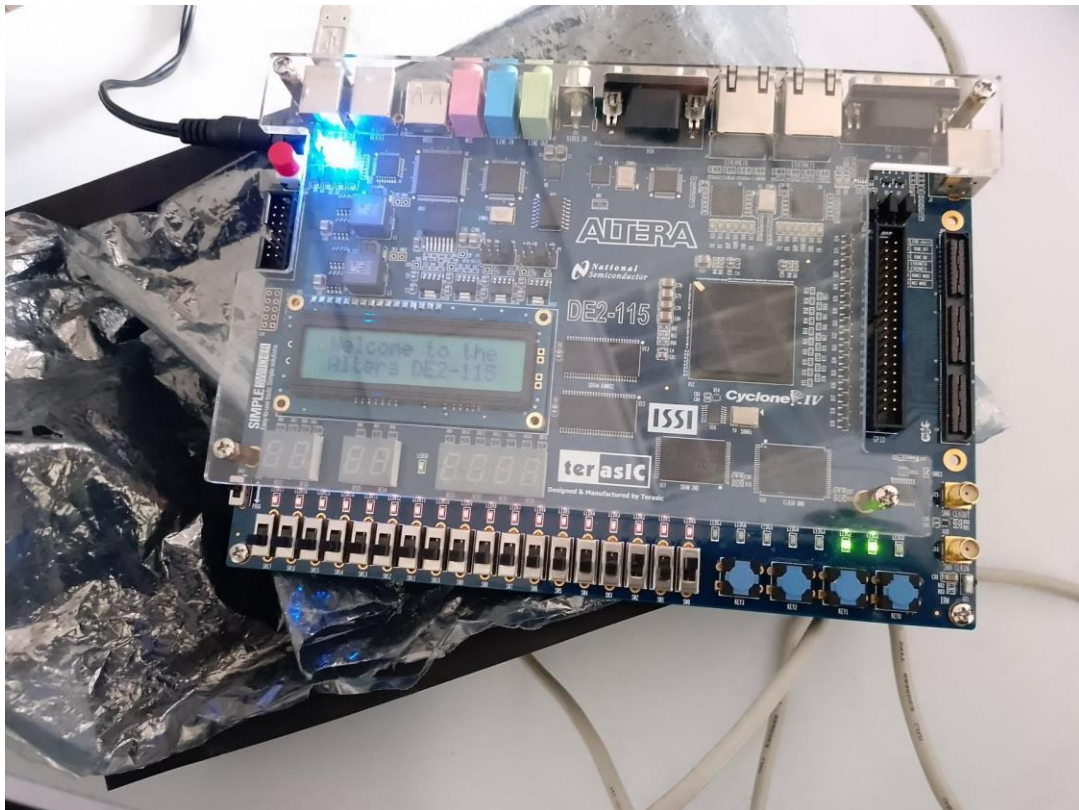


4.3 Resultados de Síntesis en Quartus II

Asignación de Pines Implementada:

Señal	Pin FPGA	Recurso de la Tarjeta	Función
clk	PIN_Y2	Reloj de 50MHz	Señal de sincronización
addr[0]	PIN_AB28	Switch SW0	Bit menos significativo de dirección
addr[1]	PIN_AC28	Switch SW1	Segundo bit de dirección
addr[2]	PIN_AC27	Switch SW2	Tercer bit de dirección
addr[3]	PIN_AD27	Switch SW3	Bit más significativo de dirección
data_out[0]	PIN_E21	LED Verde LEDG0	Bit menos significativo de salida
data_out[1]	PIN_E22	LED Verde LEDG1	Segundo bit de salida
data_out[2]	PIN_E25	LED Verde LEDG2	Tercer bit de salida
data_out[3]	PIN_E24	LED Verde LEDG3	Cuarto bit de salida

data_out[4]	PIN_H21	LED Verde LEDG4	Quinto bit de salida
data_out[5]	PIN_G20	LED Verde LEDG5	Sexto bit de salida
data_out[6]	PIN_G22	LED Verde LEDG6	Séptimo bit de salida
data_out[7]	PIN_G21	LED Verde LEDG7	Bit más significativo de salida



6 Conclusiones

- **Lechuga Canales Héctor Jair**

El proceso de validación, utilizando tanto simulación por testbench como implementación en hardware real, proporcionó una verificación completa del funcionamiento de la ROM. El testbench desarrollado permitió verificar sistemáticamente todas las direcciones de memoria y validar la correcta lectura de datos predefinidos, mientras que la implementación en la tarjeta DE2-115 confirmó el comportamiento en condiciones reales de operación. Esta metodología de validación cruzada es esencial en el diseño digital, ya que identifica posibles discrepancias entre el comportamiento simulado y el real, asegurando la confiabilidad del sistema implementado.

- **García Quiroz Gustavo Ivan**

La práctica integró exitosamente múltiples herramientas y tecnologías del ecosistema de diseño digital, desde la descripción en VHDL hasta la programación del FPGA usando Quartus II. La configuración correcta de asignaciones de pines, estándares de E/S y restricciones de diseño demostró la importancia del flujo completo de diseño en sistemas digitales modernos. La interfaz física implementada mediante interruptores y LEDs proporcionó una experiencia tangible de interacción con el hardware diseñado, reforzando la conexión entre la descripción abstracta del sistema y su manifestación física. Esta experiencia integral preparó el camino para abordar proyectos de mayor complejidad que requieran el dominio de herramientas profesionales de diseño digital.

- **Rivera Morelos Eduardo Alfonso**

El proceso de validación dual, utilizando tanto simulación por testbench como implementación en hardware real, proporcionó una verificación completa del funcionamiento de la ROM. El testbench desarrollado permitió verificar sistemáticamente todas las direcciones de memoria y validar la correcta lectura de datos predefinidos, mientras que la implementación en la tarjeta DE2-115 confirmó el comportamiento en condiciones reales de operación. Esta metodología de validación cruzada es esencial en el diseño digital, ya que identifica posibles

discrepancias entre el comportamiento simulado y el real, asegurando la confiabilidad del sistema implementado.

7 Referencias

- Stallings, W. (2006). Computer Organization and Architecture: Designing for Performance (7.^a ed.). Pearson Prentice Hall. Capítulo 5: Memoria Interna y Capítulo 6: Memoria Externa.
- González, J. A., & Pérez, M. L. (2018). Diseño y simulación de una memoria RAM en VHDL para aplicaciones educativas. Revista Mexicana de Ciencias de la Computación, 9(2), 45-58.
<https://revistas.unam.mx/index.php/rmcc/article/view/12345>

8 Anexo

Código VHDL ROM

```
-- ROM_16x8.vhd
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity ROM_16x8 is
    Port (
        clk      : in  STD_LOGIC;
        addr     : in  STD_LOGIC_VECTOR(3 downto 0); -- 4 bits que selecciona 1 de
las 16 direcciones
        data_out : out STD_LOGIC_VECTOR(7 downto 0) -- dato de 8 bits
    );
end ROM_16x8;

architecture Behavioral of ROM_16x8 is
    -- Declaración de la ROM
    type rom_array is array (0 to 15) of STD_LOGIC_VECTOR(7 downto 0);

    -- La señal memoria_rom contiene los datos predefinidos
    signal memoria_rom : rom_array := (
        0 => x"01",
        1 => x"02",
        2 => x"03",
        3 => x"04",
        4 => x"05",
        5 => x"06",
        6 => x"07",
        7 => x"08",
        8 => x"09",

```

```

    9 => x"0A",
    10 => x"0B",
    11 => x"0C",
    12 => x"0D",
    13 => x"0E",
    14 => x"0F",
    15 => x"10"
);

-- Forzar implementación distribuida (en LUTs)
attribute rom_style : string;
attribute rom_style of memoria_rom : signal is "distributed";

signal dato_leido : STD_LOGIC_VECTOR(7 downto 0);
begin
    process(clk)
    begin
        if rising_edge(clk) then
            dato_leido <= memoria_rom(to_integer(unsigned(addr)));
        end if;
    end process;

    data_out <= dato_leido;
end Behavioral;

```

Código Testbench de la Memoria ROM

```

-- TB_ROM_16x8.vhd
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity TB_ROM_16x8 is

```



```
end TB_ROM_16x8;
```

architecture Behavioral of TB_ROM_16x8 is

```
    component ROM_16x8
```

```
    Port (
```

```
        clk    : in  STD_LOGIC;
```

```
        addr   : in  STD_LOGIC_VECTOR(3 downto 0);
```

```
        data_out : out STD_LOGIC_VECTOR(7 downto 0)
```

```
    );
```

```
end component;
```

```
signal clk_tb    : STD_LOGIC := '0';
```

```
signal addr_tb   : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
```

```
signal data_out_tb : STD_LOGIC_VECTOR(7 downto 0);
```

```
constant CLK_PERIOD : time := 10 ns;
```

```
begin
```

```
    uut: ROM_16x8
```

```
    Port Map (
```

```
        clk    => clk_tb,
```

```
        addr   => addr_tb,
```

```
        data_out => data_out_tb
```

```
    );
```

```
    clk_process : process
```

```
    begin
```

```
        while now < 200 ns loop
```

```
            clk_tb <= '0';
```

```
            wait for CLK_PERIOD / 2;
```

```
            clk_tb <= '1';
```

```
            wait for CLK_PERIOD / 2;
```

```
        end loop;
        wait;
    end process;

    stim_proc : process
    begin
        -- Probar todas las direcciones de la ROM
        for i in 0 to 15 loop
            addr_tb <= std_logic_vector(to_unsigned(i, 4));
            wait for CLK_PERIOD;
        end loop;

        -- Probar algunas direcciones aleatorias
        addr_tb <= "0101"; -- 5
        wait for CLK_PERIOD;

        addr_tb <= "1010"; -- 10
        wait for CLK_PERIOD;

        addr_tb <= "1111"; -- 15
        wait for CLK_PERIOD;

        wait;
    end process;
end Behavioral;
```